Carátula para entrega de prácticas

Laboratorio de computación salas A y B

Profesor:	M.C Rene Adrian Davila Perez
Asignatura:	Programación orientada a objetos
Grupo:	7
No de Práctica(s):	9
Integrante(s):	323036285 323279866 323209432
	323161891 323063609
No. De Lista o brigada	3
Semestre:	2027-1
Fecha de entrega:	3/11/2026
Obervaciones:	

CALIFICACIÓN: _____

Índice

1. Introducción **2**

2. Marco Teórico **3**
2.1. Lenguaje Unificado de Modelado (UML) 3
2.2. Diagramas de Clases y Relaciones Estáticas 3
2.3. Diagramas de Secuencia y Comportamiento Dinámico 4
2.4. Interfaces y Clases Abstractas en Java 5

3. Desarrollo **6**
3.1. Modelado de Fecha y Empleado (Práctica 5) 6
3.2. Jerarquía de Cuadriláteros y Geometría Cartesiana (Práctica 7) 7
3.3. Contratos Abstractos y Polimorfismo de Huella de Carbono (Práctica 8) 7
3.4. Arquitectura Integral del Sistema Hospitalario (Etapa 2 del Proyecto) 8

4. Resultados **10**

5. Conclusión **14**

1. Introducción

Durante la formacion en lenguaje Java, al trabajar con la programación orientada a objetos surge la dificultad de planificar la arquitectura de un sistema antes de codificar. Si no se cuenta con un plano visual estandarizado, el desarrollo en Java suele volverse desorganizado y con un acoplamiento excesivo entre clases, complicando el flujo de ejecución del programa.

Resolver esta situación nos permitirá dominar el modelado visual en UML como una herramienta indispensable para construir software modular, limpio y bien estructurado.

Objetivos

- Elaborar diagramas de secuencia cronológicos con líneas de vida, focos de activación y paso de mensajes para validar el flujo dinámico de ejecución de los objetos en tiempo de ejecución.
- Modelar relaciones de agregación y cadenas de herencia vertical en diagramas de clases para representar la especialización de comportamientos y la reutilización de código fuente.
- Representar la implementación de contratos abstractos e interfaces de software para desacoplar el cómputo polimórfico en entidades de distinta naturaleza.
- Cotejar la correspondencia arquitectónica entre los esquemas visuales y el código fuente compilado en Java para asegurar la integridad técnica de las especificaciones.

2. Marco Teórico

En esta práctica se utilizaron los conceptos de modelado visual mediante el Lenguaje Unificado de Modelado (UML) [3], diseño estático con diagramas de clases y relaciones de orientación a objetos [2], representación dinámica con diagramas de secuencia [1], así como los contratos de interfaz [4] y la especialización con clases abstractas en Java [5].

2.1. Lenguaje Unificado de Modelado (UML)

El Lenguaje Unificado de Modelado (UML) es un estándar industrial formalizado por el Object Management Group (OMG) para la especificación, visualización, construcción y documentación de los esquemas que componen un sistema de software orientado a objetos [3].

UML proporciona una notación gráfica común e independiente de cualquier lenguaje de programación específico, permitiendo a los ingenieros modelar la arquitectura de un programa antes y durante la etapa de codificación. La taxonomía de UML se divide principalmente en dos grandes categorías de diagramas: los estructurales, que capturan los aspectos estáticos del sistema (como clases, interfaces y sus interconexiones físicas o lógicas), y los de comportamiento, que representan la dinámica temporal y el intercambio de mensajes entre objetos en tiempo de ejecución [3].

2.2. Diagramas de Clases y Relaciones Estáticas

El diagrama de clases es el esquema estructural primordial en el paradigma orientado a objetos. Modela el esquema conceptual del software al describir las clases que intervienen en el sistema, sus atributos internos, sus operaciones y los vínculos asociativos entre ellas [2].

En la notación estándar de UML, cada clase se representa mediante un rectángulo dividido en tres compartimentos horizontales: el compartimento superior contiene el nombre de la entidad; el intermedio enumera los atributos con su visibilidad (+ para público, - para privado y # para protegido) y su tipo de dato; y el compartimento inferior especifica los métodos, detallando parámetros de entrada y tipo de valor de retorno [2].

Las conexiones entre clases en UML reflejan los mecanismos de acoplamiento de la programación orientada a objetos:

- **Asociación:** Representa una relación semántica general entre dos clases, modelada con una línea continua simple con o sin flecha de navegabilidad.
- **Agregación:** Modela una relación todo-parte donde los componentes pueden subsistir independientemente del contenedor. Se representa con un rombo hueco en el extremo de la clase contenedora.
- **Composición:** Es una forma estricta de agregación donde los componentes dependen del ciclo de vida del contenedor. Se representa con un rombo relleno.
- **Generalización:** Modela la herencia entre una superclase y sus subclases. Se representa mediante una línea continua que remata en una punta de flecha triangular hueca orientada hacia la superclase.
- **Realización:** Indica que una clase implementa las operaciones estipuladas por un contrato o interfaz. Se traza con una línea discontinua con punta de flecha triangular hueca dirigida hacia la interfaz.

2.3. Diagramas de Secuencia y Comportamiento Dinámico

Los diagramas de secuencia pertenecen a la familia de diagramas de interacción de UML. Su función es modelar el comportamiento dinámico del sistema, ilustrando cómo colaboran los objetos a lo largo del tiempo para llevar a cabo un caso de uso o flujo de procesamiento específico [1].

A diferencia de los diagramas de clases, que no contemplan la variable temporal, el diagrama de secuencia organiza la interacción en dos dimensiones: el eje horizontal representa los objetos participantes con sus respectivas líneas de vida, y el eje vertical representa la progresión cronológica descendente del tiempo [1].

Los componentes formales que estructuran un diagrama de secuencia son:

- **Línea de vida:** Representada por un rectángulo en la parte superior con el nombre del objeto y una línea vertical punteada que simboliza su existencia temporal.

- **Foco de activación:** Rectángulo estrecho sobre la línea de vida que indica el periodo durante el cual el objeto procesa activamente una instrucción o ejecuta un método.
- **Mensaje síncrono:** Flecha con punta sólida que modela la llamada bloqueante a un método, donde el emisor detiene su flujo hasta recibir respuesta.
- **Mensaje de retorno:** Flecha discontinua que transporta el valor de retorno o confirma la finalización del método hacia el invocador.
- **Destrucción del objeto:** Aspa en forma de cruz (X) al final de la línea de vida que marca la liberación de recursos en memoria.

2.4. Interfaces y Clases Abstractas en Java

En el lenguaje Java, la abstracción y el desacoplamiento se fundamentan en el uso de contratos formales y plantillas de especialización [4, 5].

Una interfaz en Java (palabra reservada `interface`) establece un contrato de software 100 % abstracto que define qué operaciones debe realizar un objeto sin determinar cómo deben llevarse a cabo. A través de la palabra `implements`, cualquier clase concreta asume la obligación de proveer la implementación de todos los métodos declarados en la interfaz [4].

Por otro lado, una clase abstracta (palabra reservada `abstract class`) es un tipo no instanciable directamente que puede combinar métodos concretos completamente implementados con métodos abstractos pendientes de definición en las clases hijas. La clase abstracta se emplea mediante herencia con `extends` para modelar relaciones estrictas de especialización y permitir la reutilización de código fuente [5].

3. Desarrollo

El desarrollo técnico de la práctica consistió en la elaboración y validación de los diagramas de clases y de secuencia en UML correspondientes a los sistemas de software desarrollados a lo largo del curso en Java: Práctica 5 (encapsulamiento y constructores en `Fecha` y `Empleado`), Práctica 7 (geometría cartesiana y jerarquía de cuadriláteros), Práctica 8 (polimorfismo y contratos de interfaz con `HuellaCarbono`) y la Etapa 2 del Proyecto (arquitectura hospitalaria modular en paquetes). Todos los esquemas fueron modelados en la plataforma Draw.io garantizando una correspondencia de ingeniería 1:1 frente al código fuente compilado.

3.1. Modelado de Fecha y Empleado (Práctica 5)

En la Práctica 5 se estructuraron dos pares de clases en Java para demostrar el encapsulamiento y la manipulación de estados internos a través de constructores y métodos de acceso.

Para la entidad `Fecha`, se modelaron tres atributos privados: `mes`, `dia` y `año`. Se implementó el constructor sobrecargado `Fecha(int mes, int dia, int año)` con validaciones para restringir los valores dentro de los rangos válidos del calendario, así como el método utilitario `mostrarFecha()`, cuya signatura pública y retorno `void` despliegan la fecha formateada en consola. En el diagrama de secuencia se representó la creación de la instancia `f1` en memoria, seguida de las invocaciones a los métodos mutadores `setMes()`, `setDia()` y `setAo()`, culminando con la visualización del estado actualizado.

Para la entidad `Empleado`, se formalizó el sistema de compensaciones laborales encapsulando los campos `nombre`, `apellido` y `salarioMensual`. Se diseñó el constructor parametrizado para validar que la compensación fuera estrictamente positiva, junto con el método de negocio `getSalarioAnual()`, cuya función consistió en calcular y retornar el salario anual multiplicando el monto mensual por doce meses. En el diagrama de interacción se modeló la instanciación de dos empleados (`e1` y `e2`), la consulta inicial de sus salarios anuales, el cálculo del incremento porcentual del 10 % mediante `setSalarioMensual(salarioMensual * 1.10)` y la reevaluación del salario anual recalculado.

3.2. Jerarquía de Cuadriláteros y Geometría Cartesiana (Práctica 7)

En la Práctica 7 se abordó el modelado de una jerarquía de polígonos sobre el plano cartesiano de dos dimensiones, implementando conceptos de herencia continua, agregación de objetos de soporte y polimorfismo geométrico.

La arquitectura partió de la clase `Punto`, encargada de encapsular las coordenadas cartesianas privadas `x` e `y` con sus respectivos métodos de lectura. Posteriormente, se definió la superclase concreta `Cuadrilatero`, la cual mantiene una relación de agregación (rombo hueco) de cuatro a uno con la clase `Punto`, formalizando que la entidad agrupa cuatro instancias de vértices que delimitan geométricamente el polígono.

A partir de `Cuadrilatero`, se diseñó una cadena continua de herencia de cuatro niveles: `Cuadrilatero` es extendida por `Trapezoide`, subclase donde se introduce formalmente el método `calcularArea()`, la cual a su vez es heredada por `Paralelogramo`, de esta desciende `Rectangulo`, y finalmente `Cuadrado` culmina la especialización. Cada una de estas subclases implementa constructores especializados y redefine `calcularArea()` adaptando la fórmula geométrica a sus restricciones geométricas. En el diagrama de secuencia se modeló a `Trapezoide` como caso representativo de la jerarquía, ilustrando la instanciación de los vértices de soporte, la construcción del polígono, la invocación de `mostrarVertices()` y la resolución polimórfica del área.

3.3. Contratos Abstractos y Polimorfismo de Huella de Carbono (Práctica 8)

La Práctica 8 formalizó el desacoplamiento estructural mediante la definición de la interfaz pura `HuellaCarbono`, la cual estableció un contrato que obliga a cualquier clase participante a implementar el método público `getHuellaCarbono()`, cuyo retorno es un valor numérico de tipo `double` que representa las emisiones de dióxido de carbono en kilogramos.

Se implementaron tres entidades heterogéneas que realizan el contrato de interfaz: `Edificio`, que encapsuló `consumoElectricoKwh` y `numeroPisos` calculando sus emisiones con el factor 0.438; `Bicicleta`, que modeló el transporte no motorizado mediante `kilometrosPedaleados` con una huella nula de 0.0; y `Auto`, que encapsuló `kilometrosRecorridos` y `rendimientoKml`. En la clase `Auto` se destacó formalmente el método público `getLitrosConsumidos(): double`, el cual divide los kilómetros recorridos entre el rendimiento de com-

bustible para obtener el consumo real de gasolina, sirviendo de base matemática para multiplicar por el factor de 2.31 kg CO_2/L.

La relación entre la clase ejecutora `Main` y la interfaz se representó como una dependencia de uso («use») sin cardinalidad en la arista, reflejando que el método principal manipula las entidades a través de su abstracción. En el diagrama de secuencia se modeló el flujo de instanciación de las entidades, la ejecución de sus métodos propios (`encenderLucesPiso()`, `tocarClaxon()`, `cambiarVelocidad()`) y su incorporación en una colección homogénea `ArrayList<HuellaCarbono>` compuesta por tres instancias, sobre la cual se iteró polimórficamente para invocar `getHuellaCarbono()` y desplegar las emisiones calculadas sin requerir distinción de tipos en el ciclo principal.

3.4. Arquitectura Integral del Sistema Hospitalario (Etapa 2 del Proyecto)

La Etapa 2 del proyecto integró la totalidad de los conceptos vistos en clase mediante un simulador hospitalario universitario organizado rigurosamente en cuatro paquetes modulares: `hospital.sistema`, `personal`, `pacientes` e `interfaces`.

En el diagrama de clases integral, la clase `Sistema` se posicionó como el núcleo de gestión, manteniendo relaciones de agregación (rombo hueco) y multiplicidad de uno a muchos hacia las entidades de `Paciente` y hacia las referencias asistenciales de `AtencionMedica`, administradas internamente mediante colecciones dinámicas de tipo `ArrayList`.

Se modelaron tres jerarquías coordinadas: la jerarquía de pacientes, con la clase base abstracta `Paciente` extendida por `PacienteGeneral` y `PacienteEspecialidad`; la jerarquía médica, con la clase abstracta `Medico` especializada en `MedicoGeneral` y `MedicoEspecialista`; y la jerarquía de enfermería, con la clase abstracta `Enfermero` extendida por `EnfermeroGeneral`, `EnfermeroEspecialista` y `JefeEnfermeria`. Tanto `Medico` como `Enfermero` implementan la interfaz `AtencionMedica` cumpliendo el contrato del método `darTratamiento()`. Por su parte, la operación diagnóstica `darConsulta(Paciente p)` se modeló con fidelidad al código fuente como un método exclusivo de la jerarquía médica, garantizando la división estricta de responsabilidades clínicas.

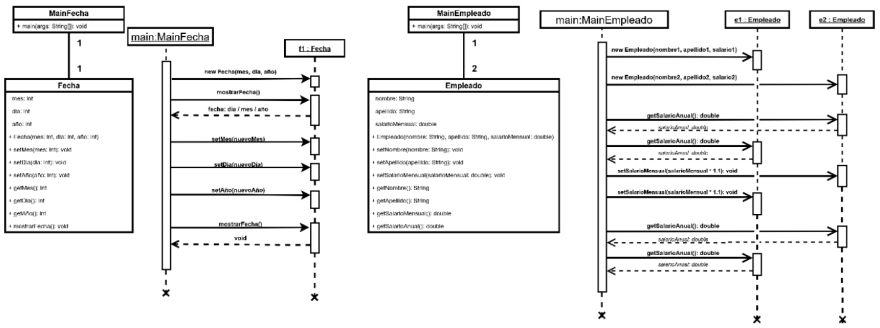
En el diagrama de secuencia correspondiente, se modeló el flujo representativo de un caso de uso asistencial completo: la admisión del paciente en el

sistema, el registro de médicos y enfermeros, la asignación clínica al médico tratante, la ejecución del diagnóstico médico y la posterior canalización al personal de enfermería para la aplicación del tratamiento y el registro final del paciente curado.

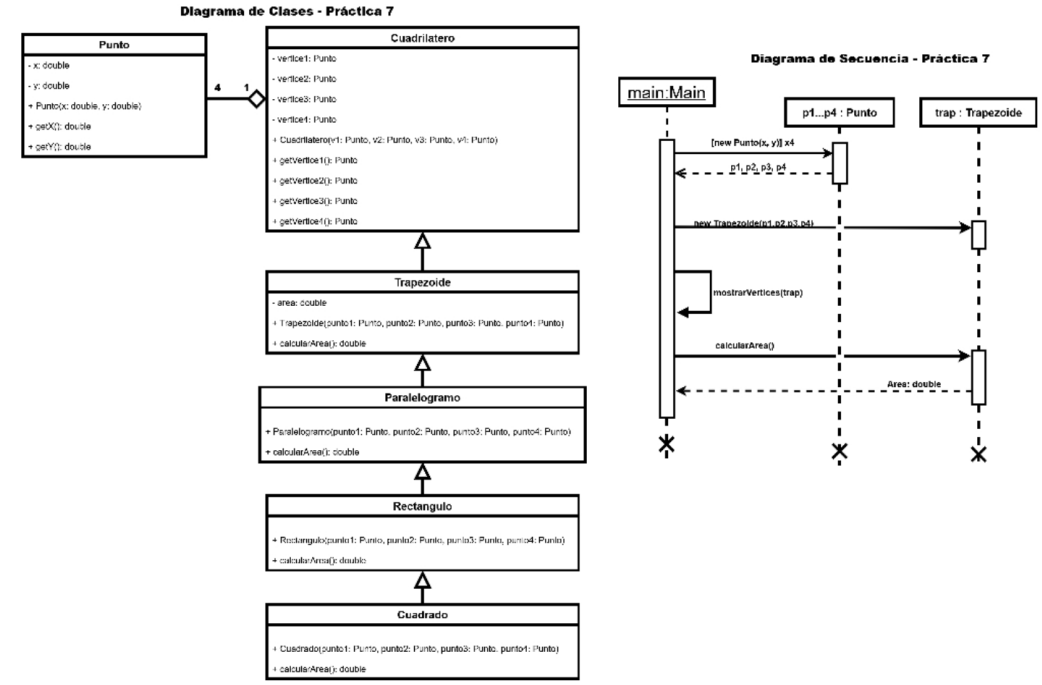
4. Resultados

Como evidencia directa del trabajo desarrollado, se obtuvieron cuatro esquemas visuales integrales en UML que modelan la arquitectura estática y el comportamiento dinámico de los proyectos implementados. Los diagramas fueron auditados letra por letra contra el código compilado para garantizar su exactitud técnica.

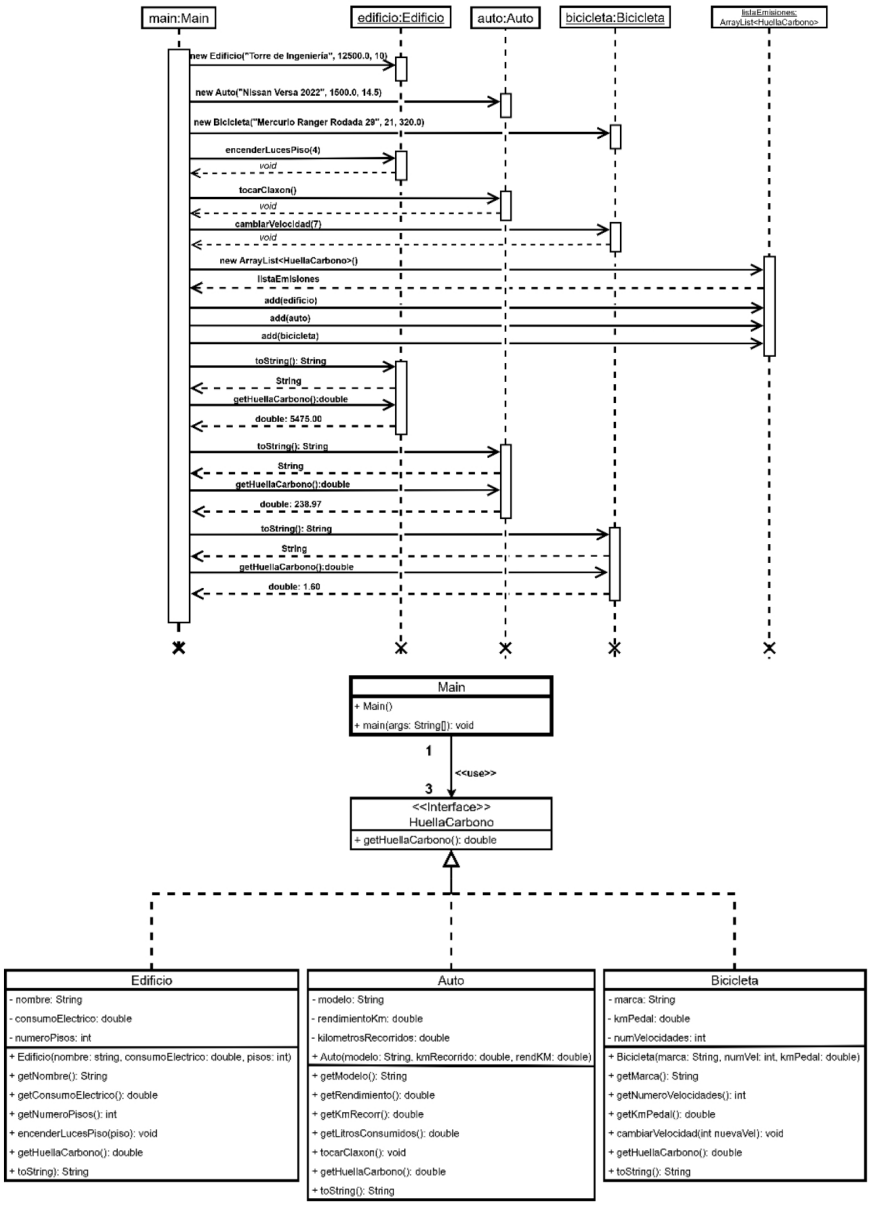
En el primer esquema se corroboró la división de responsabilidades y el encapsulamiento de datos para `Fecha` y `Empleado`, validando la interacción síncrona de los métodos mutadores y el recálculo salarial en el diagrama de secuencia:



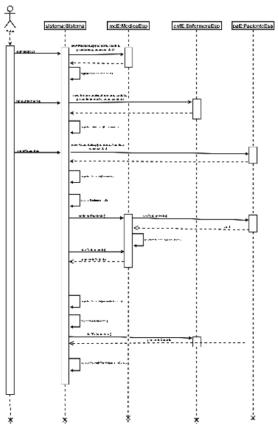
En el segundo esquema se plasmó la jerarquía continua de especialización de cuadriláteros sobre el plano cartesiano, demostrando la agregación con la clase `Punto` y la resolución polimórfica del cálculo del área en `Trapezoide`:

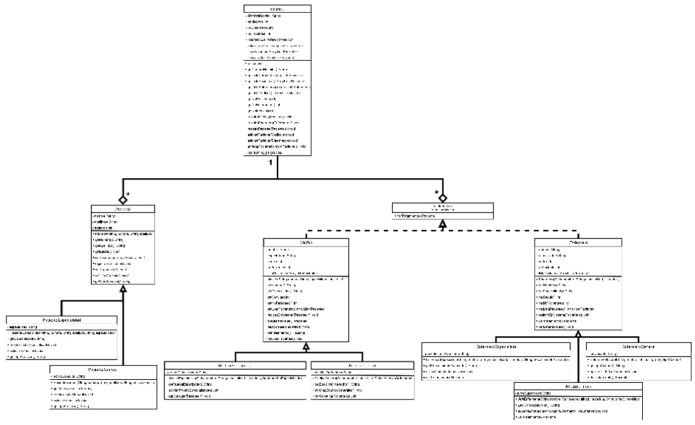


En el tercer esquema se acreditó el desacoplamiento arquitectónico mediante la interfaz `HuellaCarbono` y la dependencia con la clase `Main`, verificando la presencia del método `getLitrosConsumidos(): double` en `Auto` y la invocación polimórfica en tiempo de ejecución:



En el cuarto esquema se consolidó la arquitectura hospitalaria coordinada sobre pacientes y personal de atención médica mediante agregación y colecciones dinámicas, validando el flujo asistencial de admisión, diagnóstico y tratamiento:





5. Conclusión

En esta práctica la importancia de saber los distintos tipos de diagramas UML nos permiten saber que tipo de programa vamos a realizar antes de escribirlo , además cada uno cumple con una función diferente como los de caso de uso y los de estado y por eso es importante saber elegir al momento de resolver el problema. Estos diagramas nos ayudan a ordenar las ideas y encontrar errores para poder corregirlos antes de generar todo el código y dejar escrita la información relevante para la solución del problema.

Referencias

[1] GeeksforGeeks. *Sequence Diagram in UML*. https://www.geeksforgeeks.org/unified-modeling-language-uml-sequence-diagrams/. Accedido: 02-nov-2026. 2026.

[2] GeeksforGeeks. *Unified Modeling Language (UML) Class Diagrams*. https://www.geeksforgeeks.org/unified-modeling-language-uml-class-diagrams/. Accedido: 02-nov-2026. 2026.

[3] GeeksforGeeks. *Unified Modeling Language (UML) Introduction*. https://www.geeksforgeeks.org/unified-modeling-language-uml-introduction/. Accedido: 02-nov-2026. 2026.

[4] Oracle. *What Is an Interface?* https://docs.oracle.com/javase/tutorial/java/concepts/interface.html. The Java Tutorials, Oracle Corporation. Accedido: 02-nov-2026. 2026.

[5] W3Schools. *Java Abstract Classes and Methods*. https://www.w3schools.com/java/java_abstract.asp. Accedido: 02-nov-2026. 2026.